4.7.4 Итоговые запросы, запрос на запросе и запрос с подзапросом

Итоговые запросы 11–14, запрос на запросе 15 и запрос с подзапросом 16 берут данные из нескольких таблиц, условия отбора передаются параметрами функций.

Запрос 11 — итоговый без условия: считает пользователей каждой группы и всего; строка «Все пользователи» получена группировкой ROLLUP, а не отдельным запросом (рисунки 4.32 и 4.33); панель групп экрана «Пользователи».

```
CREATE OR REPLACE VIEW api.v_user_groups WITH (security_barrier = true) AS
SELECT CASE WHEN grouping(g.rolname) = 1 THEN 'Все пользователи'
            ELSE sec.group_title(g.rolname::text) END AS title,
       g.rolname::text AS group_name,
       count(*) AS users,
       CASE WHEN grouping(g.rolname) = 1 THEN 0
            ELSE array_position(ARRAY['grp_dba', 'grp_admin', 'grp_chief', 'grp_manager', 'grp_operator']::name[],
                                g.rolname) END AS ord
FROM sec.app_user u
JOIN pg_catalog.pg_roles r        ON r.rolname = u.login
JOIN pg_catalog.pg_auth_members m ON m.member = r.oid AND m.inherit_option
JOIN pg_catalog.pg_roles g        ON g.oid = m.roleid
WHERE g.rolname = ANY ((SELECT sec.visible_groups(sec.current_group()))::name[])
  AND (SELECT sec.assert_password_changed())
GROUP BY ROLLUP (g.rolname)
ORDER BY ord;
```

Рисунок 4.32 – Запрос 11 «Пользователи по группам» (api.v_user_groups)

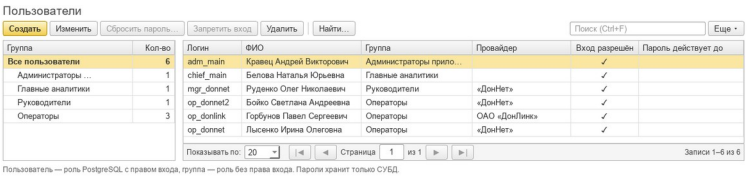
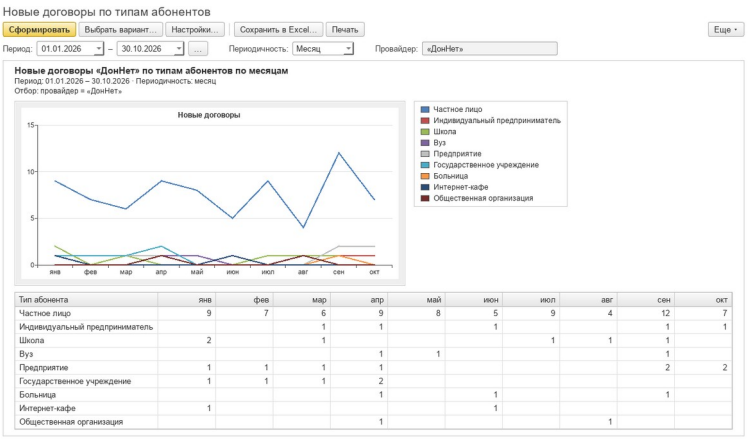


Рисунок 4.33 – Результат запроса 11: панель групп экрана «Пользователи»

Запрос 12 — итоговый с условием на данные: считает новые договоры по периодам (месяц, квартал, год) и типам абонентов; условие WHERE отбирает договоры видимых провайдеров, заключённые в периоде (рисунки 4.34 и 4.35); отчёт «Новые договоры по типам абонентов».

```
SELECT k.period, sec.period_title(p_grain, k.period), st.id::integer, st.name::text, k.contracts
FROM (SELECT date_trunc(p_grain, c.connected_on::timestamp)::date AS period, s.subscriber_type_id,
           count(*) AS contracts
      FROM core.contract c
      JOIN core.subscriber s ON s.id = c.subscriber_id
      WHERE c.provider_id = ANY (v_scope) AND c.connected_on BETWEEN p_from AND p_to
      GROUP BY 1, s.subscriber_type_id) k
JOIN ref.subscriber_type st ON st.id = k.subscriber_type_id
ORDER BY k.period, st.id;
```

Рисунок 4.34 – Запрос 12 «Новые договоры по периодам и типам абонентов» (api.report_new_contracts)



Тип абонента	янв	фев	мар	апр	май	июн	июл	авг	сен	окт
Частное лицо	9	7	6	9	8	5	9	4	12	7
Индивидуальный предприниматель			1	1		1			1	1
Школа	2		1				1	1	1	
Вуз				1	1				1	
Предприятие	1	1	1	1					2	2
Государственное учреждение	1	1	1	2						
Больница				1		1			1	
Интернет-кафе	1					1				
Общественная организация				1				1		

Рисунок 4.35 – Результат запроса 12 в отчёте «Новые договоры по типам абонентов»

Запрос 13 — итоговый с условием на группы: выбирает общих абонентов — тех, у кого договоры не меньше чем с заданным числом провайдеров; условие записано в HAVING (рисунки 4.36 и 4.37); экран «Общие абоненты» администратора приложения.

```
SELECT s.id, s.name::text, st.name::text, ci.name::text, s.address::text, s.phone::text,
       count(DISTINCT c.provider_id), count(*) OVER ()
FROM core.subscriber s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN ref.city ci            ON ci.id = s.city_id
JOIN core.contract c        ON c.subscriber_id = s.id
GROUP BY s.id, st.id, ci.id
HAVING count(DISTINCT c.provider_id) >= p_min_providers
ORDER BY s.name, s.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.36 – Запрос 13 «Общие абоненты» (api.q_shared_subscribers)

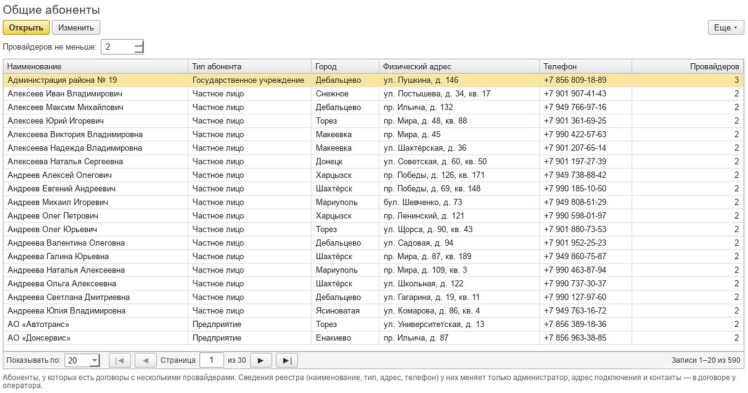


Рисунок 4.37 – Результат запроса 13 на экране «Общие абоненты»

Запрос 14 — итоговый с условием на данные и на группы: условие WHERE отбирает действующие договоры провайдера, условие HAVING — договоры, у которых последний сеанс раньше заданного числа дней (рисунки 4.38 и 4.39); счётчик начальной страницы и список «Договоры без сеансов».

```
SELECT c.id, c.contract_no::text, s.id, s.name::text, coalesce(c.contact_phone, s.phone)::text,
       c.connected_on, max(se.started_at),
       current_date - greatest(max(se.started_at)::date, c.connected_on),
       count(*) OVER ()
FROM core.contract c
JOIN core.subscriber s    ON s.id = c.subscriber_id
LEFT JOIN core.session se ON se.contract_id = c.id
WHERE c.provider_id = ANY (v_scope) AND c.closed_on IS NULL
  AND c.connected_on < v_border
GROUP BY c.id, c.provider_id, s.id
HAVING max(se.started_at) IS NULL
    OR max(se.started_at) < v_border::timestamptz
ORDER BY max(se.started_at) NULLS FIRST, c.connected_on, c.contract_no
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.38 – Запрос 14 «Договоры без сеансов» (api.q_idle_contracts)

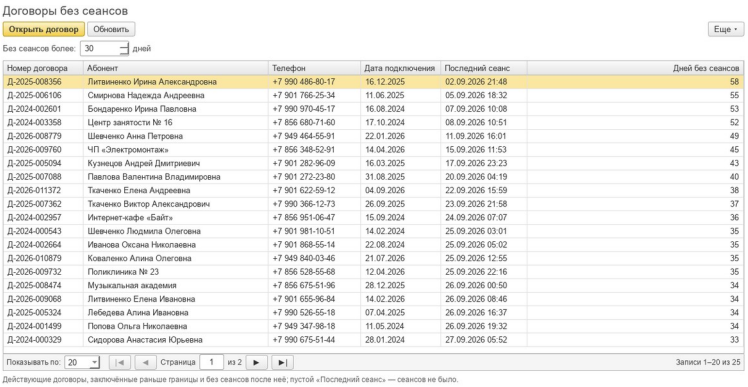


Рисунок 4.39 – Результат запроса 14 на экране «Договоры без сеансов»

Запрос 15 — запрос на запросе по принципу итогового запроса. Внутренний запрос — представление доходных операций (запрос 20): плата за подключение и оплата трафика (рисунок 4.40); внешний группирует операции по

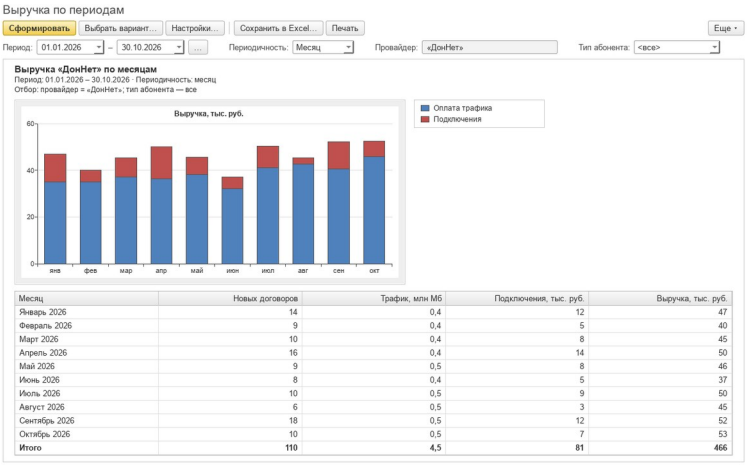
периодам с ROLLUP и добавляет строку «Итого» (рисунки 4.41 и 4.42); отчёт «Выручка по периодам» и счётчик «Выручка за месяц» начальной страницы.

```
CREATE OR REPLACE VIEW api.v_revenue_items WITH (security_barrier = true) AS
SELECT c.provider_id,
       c.subscriber_id,
       s.subscriber_type_id,
       c.connected_on::timestamptz AS at,
       'подключение'::text AS kind,
       c.connection_fee AS amount,
       0 AS volume_mb
FROM core.contract c
JOIN core.subscriber s ON s.id = c.subscriber_id
WHERE c.provider_id = ANY ((SELECT sec.scope_data())::integer[])
UNION ALL
SELECT c.provider_id,
       c.subscriber_id,
       sb.subscriber_type_id,
       se.started_at,
       'трафик'::text,
       se.cost,
       se.volume_mb
FROM core.session se
JOIN core.contract c ON c.id = se.contract_id
JOIN core.subscriber sb ON sb.id = c.subscriber_id
WHERE c.provider_id = ANY ((SELECT sec.scope_data())::integer[]);
```

Рисунок 4.40 – Запрос 20 «Доходные операции» (api.v_revenue_items) — внутренний запрос запроса 15

```
SELECT k.period, CASE WHEN k.is_total THEN 'Итого' ELSE sec.period_title(p_grain, k.period) END,
       k.new_contracts, k.volume_mb, k.connection_fees, k.revenue - k.connection_fees, k.revenue
FROM (SELECT date_trunc(p_grain, r.at)::date AS period,
             grouping(date_trunc(p_grain, r.at)::date) = 1                  AS is_total,
             count(*) FILTER (WHERE r.kind = 'подключение')                 AS new_contracts,
             coalesce(sum(r.volume_mb), 0)                                  AS volume_mb,
             coalesce(sum(r.amount) FILTER (WHERE r.kind = 'подключение'), 0)  AS connection_fees,
             coalesce(sum(r.amount), 0)                                     AS revenue
      FROM api.v_revenue_items r
      WHERE r.at >= p_from AND r.at < p_to + 1
        AND (p_provider_id IS NULL OR r.provider_id = p_provider_id)
        AND (p_subscriber_type_id IS NULL OR r.subscriber_type_id = p_subscriber_type_id)
      GROUP BY ROLLUP (date_trunc(p_grain, r.at)::date)) k
ORDER BY k.is_total, k.period;
```

Рисунок 4.41 – Запрос 15 «Выручка по периодам» (api.report_revenue)



Месяц	Новых договоров	Трафик, млн Мб	Подключения, тыс. руб.	Выручка, тыс. руб.
Январь 2026	14	0,4	12	47
Февраль 2026	9	0,4	5	40
Март 2026	10	0,4	8	45
Апрель 2026	16	0,4	14	50
Май 2026	9	0,5	8	46
Июнь 2026	8	0,4	5	37
Июль 2026	10	0,5	9	50
Август 2026	6	0,5	3	45
Сентябрь 2026	18	0,5	12	52
Октябрь 2026	10	0,5	7	53
Итого	**110**	**4,5**	**81**	**466**

Рисунок 4.42 – Результат запроса 15 в отчёте «Выручка по периодам» (руководитель)

Запрос 16 — запрос с подзапросом: отбор «Без сотрудников» в функции api.q_providers выбирает провайдеров, коды которых не входят (NOT IN) в подзапрос по сотрудникам; значения NULL в подзапросе исключаются явно, иначе разность множеств была бы пустой (рисунки 4.43 и 4.44).

```
RETURN QUERY
SELECT v.id, v.display_name, v.ownership_type, v.city, v.address, v.phone, v.founded_year,
      coalesce(k.active, 0), coalesce(k.new_in_period, 0),
      count(*) OVER ()
FROM api.v_providers v
…
WHERE (v_like IS NULL OR lower(v.name) LIKE v_like)
  AND (NOT p_without_new OR k.provider_id IS NULL OR k.new_in_period = 0)
  AND (NOT p_without_staff
      OR v.id NOT IN (SELECT u.provider_id FROM sec.app_user u WHERE u.provider_id IS NOT NULL))
ORDER BY k.active DESC NULLS LAST, v.name
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.43 – Запрос 16 «Провайдеры без сотрудников» (api.q_providers, отбор p_without_staff)

Пельмени домашние (около 10 порций, 140–150 штук)

Тесто: мука пшеничная — 500 г; молоко — 200 мл; яйца — 2 шт. (около 120 г); растительное масло — 1 ст. л.; соль — 1 ч. л.
Фарш: свино-говяжий фарш — 500 г; репчатый лук — 1 шт.; чеснок — 2 зубчика; молоко — 2 ст. л.; зелень — половина пучка; соль и перец — по вкусу.

1. Соедините всё для теста и вымешивайте около 5 минут, пока оно не станет гладким.
2. Заверните тесто в плёнку и дайте ему полежать 40 минут.
3. Мелко нарубите лук и зелень, раздавите чеснок, смешайте с фаршем и молоком, посолите и поперчите.
4. Раскатайте тесто тонким пластом (около 1 мм), разложите фарш и слепите пельмени руками или в пельменнице.
5. Опустите пельмени в кипящую подсоленную воду и варите 6 минут после того, как вода снова закипит.

Источник: журнал «Т—Ж», t-j.ru/pelmeni

Рисунок 4.44 – Результат запроса 16: провайдеры без сотрудников (администратор приложения)